Pseudo-Last and Pseudo-Last-Conditioned GShell Experiments

Internal Progress Note

June 10, 2026

Abstract

This note summarizes the code experiments I performed in `FootShellGaussian` to construct a pseudo shoe last from a neutral SUPR foot and to use that pseudo-last as an interior prior for GShell SDF/mSDF training. My goal is not to claim that the generated last is a manufacturing ground-truth last. My goal is to build a reasonable pseudo-ground-truth interior scaffold that can tell GShell which regions should plausibly remain empty for the foot and which lower support regions should plausibly remain shoe material. I also record the methods that failed, the current cleaned implementation, and an important bug I discovered late in the cross-section selector.

Contents

1 High-Level Motivation

GShell reconstructs a shoe mainly from external images. This gives strong supervision on the visible outside surface, but it gives weak or no supervision on hidden interior geometry. For shoes, that hidden region matters. The shoe should have an internal cavity where a foot could fit, and the lower sole / support region should not collapse into an infinitely thin or aggressively cut surface.

I therefore introduced a pseudo-last pipeline. The pseudo-last is not rendered as part of the shoe and is not treated as exact physical shoe material. Instead, it is used as an approximate internal reference:

- points inside the pseudo-last represent approximate empty foot cavity;
- points near and below the lower last represent plausible lower shoe support / sole / sidewall material;
- high upper regions, tongue, collar, padding, sock liner, and style details remain uncertain and should not be over-constrained.

The key distinction is:

$$\text{pseudo-last} \neq \text{exact shoe interior.}$$

It is a formula-based scaffold derived from the aligned SUPR foot and the shoe-specific support footprint.

2 Coordinate and Sign Conventions

2.1 Shoe Coordinate Convention

The current canonical GShell shoe convention used by the foot-prior code is:

$$X = \text{heel-to-toe length}, \qquad Y = \text{vertical bottom/opening axis}, \qquad Z = \text{left-right width}.$$

In the current code, `shoe_up_sign = -1`. This means that moving toward the physical opening/up direction corresponds to decreasing the stored Y coordinate. Moving toward the sole/bottom corresponds to increasing the stored Y coordinate.

2.2 SDF Sign Convention

There are two SDF-like objects in the code, and their sign conventions must be kept clear.

In the equations below, $\mathbf{p} = (x, y, z)$ means one 3D point in shoe/world coordinates. You can read $\mathbf{p}$ as "the point being tested."

Pseudo-last SDF on disk. The stored pseudo-last SDF file, `pseudo_last_sdf.npz`, follows the standard geometry convention:

$$d_L(\mathbf{p}) < 0 \quad \text{inside the pseudo-last,}$$

$$d_L(\mathbf{p}) > 0 \quad \text{outside the pseudo-last.}$$

Pseudo-last cavity field in the cleaned code. Inside `pseudo_last_loss.py`, the stored SDF is immediately flipped:

$$C_L(\mathbf{p}) = -d_L(\mathbf{p}).$$

Therefore:

$$C_L(\mathbf{p}) > 0 \quad \text{inside the pseudo-last cavity,}$$

$$C_L(\mathbf{p}) < 0 \quad \text{outside the pseudo-last cavity.}$$

This sign flip was introduced to reduce mental inversion errors. The pseudo-last field is now interpreted in the same positive-inside style used by the GShell training field, but the represented volume is different: for the pseudo-last, the positive region is the empty foot cavity; for the shoe SDF, positive means inside the learned shoe material / shell volume.

GShell shell SDF queried by the prior. The cleaned pseudo-last prior queries the learned shell SDF through:

$$S_\theta(\mathbf{p}) = \texttt{sdf\_net}(\mathbf{p}).$$

The prior uses this convention:

$$S_\theta(\mathbf{p}) > 0 \quad \text{inside learned shoe material / shell volume,}$$

$$S_\theta(\mathbf{p}) < 0 \quad \text{outside learned shoe material / shell volume.}$$

2.3 mSDF Meaning

The mSDF is the GShell field that controls which parts of the watertight shell are kept closed and which parts are cut open. In the foot-prior experiments, the intended direction was:

$$M(\mathbf{p}) > 0 \quad \text{keep / closed direction,}$$

$$M(\mathbf{p}) < 0 \quad \text{cut / open direction.}$$

The word "field" means values stored or evaluated over the tetrahedral grid. For example, the SDF field is a scalar value at grid vertices or queried points, and the mSDF field is another scalar value that controls GShell's opening/cutting behavior.

3 Notation Guide

This section is intentionally simple. It explains the symbols before the longer method description. The whole pipeline can be understood as repeatedly asking: "where is this 3D point, and should the model treat it as shoe material, empty foot cavity, or uncertain boundary?"

Symbol	Plain-English meaning
$\mathbf{p} = (x, y, z)$	A 3D point being tested. The code may ask whether this point is inside the pseudo-last, inside the learned shoe shell, or near the lower support region.
x, y, z	The coordinates of that point. x is shoe length, y is the vertical bottom/opening axis, and z is shoe width.

Symbol	Plain-English meaning
x_i	One sampled slice position along the shoe length. A section at x_i is a 2D Y-Z cross-section.
x_0, x_1	The first and last X positions of the pseudo-last section grid. They are used to normalize length.
s or s_{raw}	Normalized length position: $$s_{\text{raw}} = \frac{x - x_0}{x_1 - x_0}.$$ Near 0 means heel side, near 1 means toe side.
$c(x)$	The centerline of the shoe support footprint at length position x. It is the middle Z location between the left and right support boundaries.
$z_L(x), z_R(x)$	The left and right support boundaries in the width direction. These come from the projected bottom/support footprint.
$w_S(x)$	The support width: $$w_S(x) = z_R(x) - z_L(x).$$ It indicates how wide the shoe support footprint is at length position x.
$F(x, z)$	The estimated lower floor/support height from the reconstructed shoe geometry.
$B(x, z)$	The pseudo-footbed height. It is the smoothed and offset version of $F(x, z)$, placed toward the inside of the shoe.
$B_L(x, u_L)$	The bottom curve of the pseudo-last section used by the loss code. It is stored in `bottom_sections`. You can think of it as: "at this length position and this left-right position, where is the bottom of the pseudo-last?"
$d_L(\mathbf{p})$	The stored pseudo-last SDF. On disk it follows the standard sign convention: negative inside the pseudo-last and positive outside.
$C_L(\mathbf{p})$	The pseudo-last cavity field used inside the cleaned prior: $$C_L(\mathbf{p}) = -d_L(\mathbf{p}).$$ Positive means the point is inside the pseudo-last cavity.
ϵ_L	A small safety band around the pseudo-last surface. I use it to avoid making hard inside/outside decisions for points exactly on the boundary. In code this is `xsec_surface_band`, currently 0.003.
$S_\theta(\mathbf{p})$	The learned GShell shell SDF queried at point $\mathbf{p}$. Positive means the point is inside the learned shoe shell/material volume. Negative means it is outside.

Symbol	Plain-English meaning
$M(\mathbf{p})$ or M_i	The learned mSDF value. Positive is the intended keep/closed direction, and negative is the cut/open direction. M_i means the value at tet-grid vertex i.
$\mathbf{v}_i$	The i-th vertex of the tetrahedral grid. This is a fixed grid sample used by GShell before the mesh is extracted. It is different from $\mathbf{p}$, which means any query point.
$a(x)$	The target pseudo-last half-width at length position x. It controls how far the last extends left and right from the centerline.
$a_L(x), a_S(x)$	Half-widths used by the loss. $a_L(x)$ is the pseudo-last half-width, and $a_S(x)$ is the support-footprint half-width.
u_L, u_S	Normalized lateral coordinates. $u_L = 0$ means the point is on the last centerline. $\lvert u_L\rvert \approx 1$ means the point is near the last side. u_S does the same thing but relative to the support footprint.
h	Vertical height relative to the pseudo-last bottom: $$h = B(x, z) - y.$$ In this coordinate system, $h > 0$ is above the bottom surface, $h = 0$ is on the bottom surface, and $h < 0$ is just below it.
$\hat{h}$	Normalized height: $$\hat{h} = \frac{h}{H(x)}.$$ This indicates whether a point is near the bottom, lower sidewall, or high upper region relative to the local last height.
$H(x)$	The target pseudo-last section height at length position x.
$p_{\text{ell}}, q_{\text{ell}}$	The superellipse exponents used only for shaping pseudo-last cross-sections. They control how round or box-like a section is. They are not 3D points.
$\mathcal{E}$	The set of points classified as empty cavity. These are points inside the pseudo-last. For these points, the shoe SDF should become negative.
$\mathcal{M}$	The set of points classified as lower material/support. These are local points outside the pseudo-last but near the bottom, plantar, or sidewall regions. For these points, the shoe SDF should become positive.
$\mathcal{S}_{\text{sole}}$	Lower points just below the last bottom. These represent the region where sole support should plausibly exist.
$\mathcal{S}_{\text{plantar}}$	Points just outside the lower plantar/bottom part of the pseudo-last.
$\mathcal{S}_{\text{sidewall}}$	Points near the lower sides of the pseudo-last.
d_{sole}	The maximum allowed distance below the last bottom for a point to count as a local sole-support point. This cap was important because without it the code selected points far below the shoe.

Symbol	Plain-English meaning
$r_{\text{plantar}}, r_{\text{wall}}$	Height ratios. They say how low a point must be, relative to the local last height, to count as plantar support or lower sidewall support.
δ_S	A small extra width allowance around the support footprint. It prevents the selector from being too brittle at the support boundary.
N_x, N_y, N_z	The number of sampled supervision points along length, vertical direction, and width direction. The dense cross-section pool has $N_x N_y N_z$ candidate points before filtering.
∂L	The surface of the pseudo-last. Points on ∂L are boundary points between cavity and non-cavity.
w_i	The weight assigned to tet-grid vertex i in the mSDF keep loss. Larger weight means the prior cares more about that grid vertex.
$\Delta \mathbf{v}_i$	The learned deformation offset for tet-grid vertex i. The deformed grid vertex is $\mathbf{v}_i^{\text{def}} = \mathbf{v}_i + \Delta \mathbf{v}_i$.
$m_{\text{mat}}, m_{\text{empty}}, m_M$	Small target margins in the losses. They prevent the model from merely touching zero; they ask it to be confidently positive or negative by a small amount.
L_*	The pseudo-last loss terms: L_{mat}, L_{empty}, L_{surf}, and L_{mSDF}. They respectively supervise lower material, empty cavity, approximate last surface, and mSDF keep behavior.
λ_*	Loss weights: λ_{mat}, λ_{empty}, λ_{surf}, and λ_{mSDF}. They decide how strongly each pseudo-last loss term contributes to training.
$\alpha(t)$	The schedule/warmup value at training iteration t. It ramps the pseudo-last prior from zero strength to full strength.

One-sentence version. If $\mathbf{p}$ is inside the pseudo-last, the code treats it as empty foot cavity. If $\mathbf{p}$ is just outside and below/around the lower pseudo-last, the code treats it as plausible shoe support material. If $\mathbf{p}$ is near the pseudo-last surface, the code treats it as an approximate internal boundary rather than a hard truth.

4 Part 1: Pseudo Support Footbed

4.1 Input Meshes

For each shoe, the foot-aware alignment pipeline uses:

- `mesh_watertight/mesh.obj`: the watertight GShell shoe mesh;
- `mesh/mesh.obj`: the open/non-watertight GShell mesh.

The support footprint is estimated in `FootShellGaussian/foot_prior/support_footprint.py`. The open mesh is especially useful because it gives a lower open bottom silhouette. The watertight mesh is used as the reference mesh for lower support geometry and height estimation.

4.2 Support Footprint Extraction

The main idea is:

$$\begin{aligned}&\text{reconstructed lower shoe geometry}\\ &\rightarrow \text{2D outsole/support footprint in } XZ\\ &\rightarrow \text{centerline and width profile.}\end{aligned}$$

The extractor projects lower support samples into the X-Z plane, fills holes, keeps the outer bottom footprint, and extracts:

$$c(x) = \text{centerline in width direction,}$$

$$z_L(x), z_R(x) = \text{left and right support boundaries,}$$

$$w_S(x) = z_R(x) - z_L(x) = \text{support width profile.}$$

The usable footbed area is then approximated by shrinking the full bottom footprint inward. This is controlled by parameters such as `footbed_inner_margin_cells`. The reason is that the outsole outline is wider than the actual usable footbed area.

4.3 Pseudo-Footbed Height

The floor/support height map is converted into a pseudo-footbed height map by offsetting it toward the shoe interior:

$$B(x, z) = F(x, z) + \text{interior offset,}$$

where $F(x, z)$ is the estimated lower floor/support surface. In code, the sign of the offset is handled by `shoe_up_sign`. The important idea is that the pseudo-footbed is placed above the outsole/bottom surface, not exactly on the raw reconstructed bottom.

The pseudo-last builder later uses the smoothed version:

$$B(x, z) = \texttt{smooth\_footbed\_heightmap}(x, z).$$

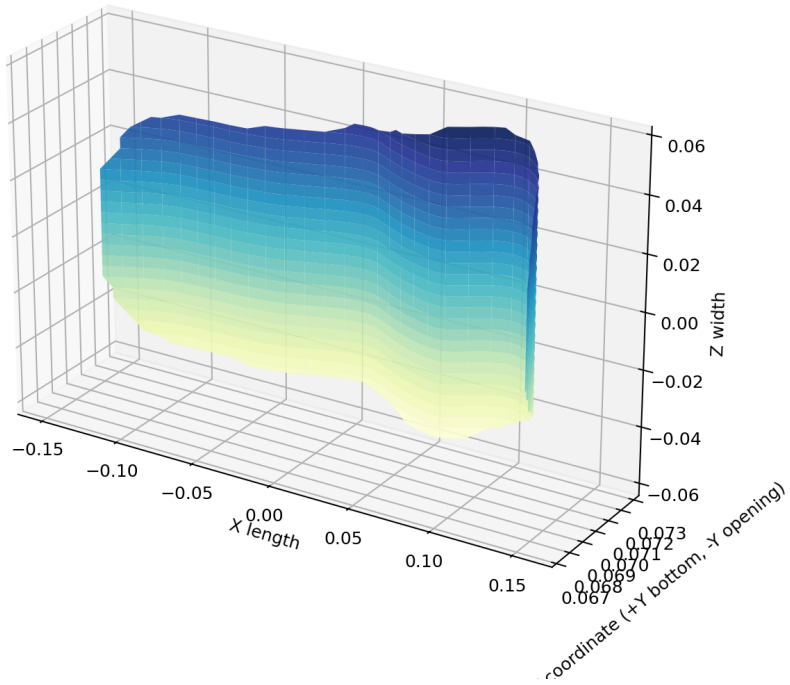

Figure 1: Air Jordan pseudo-footbed preview. This is not a scanned insole. It is a smoothed, offset support surface estimated from the reconstructed lower shoe geometry.

5 Part 2: SUPR Foot Alignment

5.1 Neutral SUPR Foot Template

The current pipeline does not estimate SUPR shape parameters per shoe. It uses a fixed neutral right-foot mesh exported from SUPR:

`baselines/SUPR/output/debug_playground/supr_male_right_foot_neutral.obj`.

Conceptually, the exported mesh corresponds to:

$$\text{pose} = 0, \qquad \text{betas} = 0, \qquad \text{trans} = 0.$$

Thus, the SUPR stage provides a generic anatomical foot template. Shoe-specific adaptation happens later through rigid/scale alignment and fitting, not through SUPR pose or beta optimization.

5.2 Initial Alignment

The initial alignment maps raw SUPR coordinates into shoe coordinates. The main scale is:

$$s_0 = \frac{\texttt{length\_ratio} \cdot L_{\text{shoe}}}{L_{\text{SUPR foot}}}.$$

In the current final alignment pipeline:

$$\texttt{length\_ratio} = 0.85.$$

The foot transform is essentially:

$$p_{\text{shoe}} = T_{\text{shoe}} R_{\text{yaw,pitch,roll}} S(s_0) T_{\text{foot}}^{-1} A_{\text{axis}} p_{\text{SUPR}},$$

where A_{axis} remaps SUPR foot axes to shoe axes.

This equation should be read from right to left. First, the raw SUPR foot point p_{SUPR} is converted into the shoe axis convention by A_{axis}. Then T_{foot}^{-1} moves the foot so its own reference point is centered. Then $S(s_0)$ scales the foot to the shoe length. Then $R_{\text{yaw,pitch,roll}}$ rotates the foot. Finally, T_{shoe} places the foot inside the shoe coordinate frame.

5.3 Optimization-Based Foot Fit

After the initial placement, the optimizer refines only:

$$\text{uniform scale}, \quad \text{yaw}, \quad \text{pitch}, \quad \text{roll}, \quad t_x, t_y, t_z.$$

The optimizer tries to:

- keep the foot inside the support footprint;
- keep plantar points near the pseudo-footbed;
- maintain reasonable heel and toe gaps;
- keep the foot centerline aligned with the support centerline.

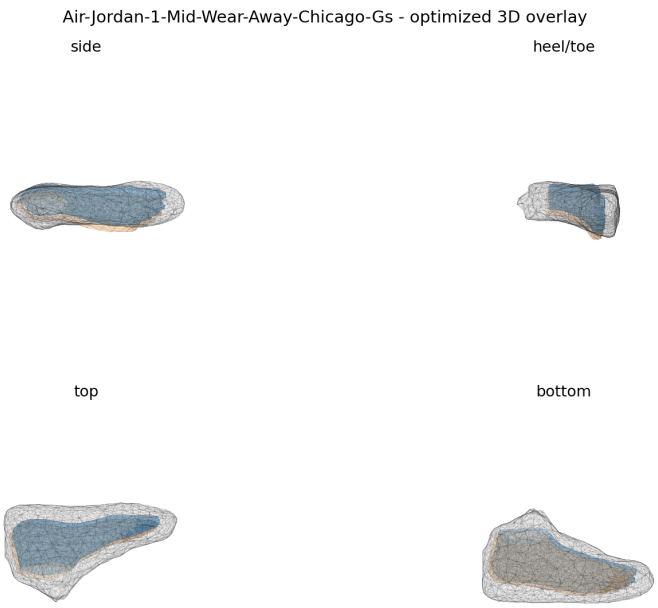

Figure 2: Air Jordan optimized SUPR foot placement over the reconstructed shoe geometry. This is the aligned foot used as input to pseudo-last construction.

6 Part 3: Formula-Based Pseudo-Last Construction

6.1 Why Section Lofting

The final pseudo-last builder does not simply inflate the SUPR foot surface. Instead, it builds a sequence of smooth cross-sections along the shoe length and lofts them into a watertight mesh. In plain language, this means:

draw a smooth last outline at many X positions $\rightarrow$ connect those outlines into a 3D surface.

Each slice is a Y-Z cross-section at fixed $X = x_i$. This gives much more control than direct surface inflation because each section has explicit width, height, bottom, arch, heel, and toe-box behavior.

6.2 Section Sampling

For each x_i, the code intersects the aligned SUPR foot mesh with the plane:

$$X = x_i.$$

If the exact triangle-plane intersection is too sparse, it falls back to nearby vertices in a small slab around x_i.

For each section point $\mathbf{p} = (x, y, z)$, the local coordinates are:

$$\xi = z - c(x),$$

$$h = B(x, z) - y.$$

Here:

- $c(x)$ is the support centerline;
- $B(x, z)$ is the pseudo-footbed height;
- h measures height above the pseudo-footbed under the current shoe coordinate convention.

6.3 Raw Section Measurements

For each x_i, the builder estimates:

$$w_{\text{raw}}(x_i) = \text{raw foot section width},$$

$$H_{\text{raw}}(x_i) = \text{raw foot section height},$$

$$\ell_{\text{plantar}}(x_i) = \text{plantar lift relative to the footbed}.$$

The toe region is handled specially. For normalized length

$$s = \frac{x - x_0}{x_1 - x_0},$$

the builder starts merging toe components at:

$$s = 0.74$$

and reaches full toe-box merging by:

$$s = 0.88.$$

This uses binary closing on the rasterized section mask, so individual toe gaps are converted into one smoother toe-box-like component.

6.4 Support-Width Clamp

The pseudo-last is clamped inside a shrunken support footprint. If

$$w_S(x) = z_R(x) - z_L(x),$$

then the inner support boundaries are:

$$z_L^{\text{inner}}(x) = c(x) - \frac{1}{2}\eta_s w_S(x),$$

$$z_R^{\text{inner}}(x) = c(x) + \frac{1}{2}\eta_s w_S(x),$$

with:

$$\eta_s = 0.93.$$

The target half-width is:

$$a(x) = \min\left(\frac{1}{2}\eta_s w_S(x), \frac{1}{2}w_{\text{raw}}(x) + \Delta_{\text{side}}(x)\right).$$

The side clearance term Δ_{side} changes by region:

$$\Delta_{\text{side}}(x) = w_{\text{raw}}(x)\left(0.012\, w_{\text{heel}}(x) + 0.020\, w_{\text{mid}}(x) + 0.040\, w_{\text{fore}}(x)\right).$$

Here w_{heel}, w_{mid}, and w_{fore} are not widths. They are smooth region weights. They decide whether the current slice behaves more like heel, midfoot, or forefoot. The side clearance is small in the heel, larger in the midfoot, and largest in the forefoot. The exact region weights are smoothstep functions over normalized length.

6.5 Height and Bottom Surface

The target height is:

$$H(x) = H_{\text{raw}}(x)\left(1 + \Delta_{\text{vamp}}(x) + \Delta_{\text{toe}}(x)\right),$$

where the current defaults are:

$$\Delta_{\text{vamp}} \le 0.05, \qquad \Delta_{\text{toe}} \le 0.04.$$

The bottom of the last follows the pseudo-footbed, but preserves a moderated SUPR arch lift:

$$y_b(x,z) = B(x,z) - \rho_{\text{arch}}(s)\, L_{\text{plantar}}(x,z).$$

Here $y_b(x,z)$ is the bottom surface of the pseudo-last, $B(x,z)$ is the pseudo-footbed from the shoe support geometry, and $L_{\text{plantar}}(x,z)$ is the amount of arch/plantar lift estimated from the aligned SUPR foot. The subtraction appears because of the current shoe coordinate convention: moving toward the physical opening/up direction means decreasing stored Y.

The arch weight is:

$$\rho_{\text{arch}}(s) = 0.75 \cdot \exp\left(-\frac{(s - 0.45)^2}{2(0.12)^2}\right).$$

In code, the lift is a blend of a scalar plantar lift and a smoothed lateral plantar lift curve. This keeps heel and toe mostly footbed-following while allowing a controlled midfoot arch.

6.6 Superellipse Section Shape

Each section is generated as a smooth superellipse-like profile. Let:

$$r = \left| \frac{z - c(x)}{a(x)} \right|.$$

The upper height profile is:

$$h_{\text{top}}(x, z) = H(x) \left(1 - r^{p_{\text{ell}}}\right)^{1/q_{\text{ell}}},$$

with:

$$p_{\text{ell}} = 3.5, \qquad q_{\text{ell}} = 2.2.$$

Near the toe, p_{ell} is increased slightly to make the toe-box smoother and more box-like. The upper curve is placed as:

$$y_{\text{top}}(x, z) = y_b(x, z) - h_{\text{top}}(x, z).$$

6.7 Heel Hold and Toe Allowance

The hindfoot upper is slightly narrowed:

$$z_{\text{new}} = c(x) + (z - c(x))\,(1 - \alpha_{\text{heel}})\,.$$

The current heel-hold strength is small:

$$\texttt{heel\_hold\_strength} = 0.05.$$

The anatomical toe is extended by:

$$L_{\text{toe ext}} = 0.05\, L_{\text{foot}},$$

clipped to the available support footprint. The extension tapers with a rounded profile:

$$\text{taper}(u) = \sqrt{1 - u^2}.$$

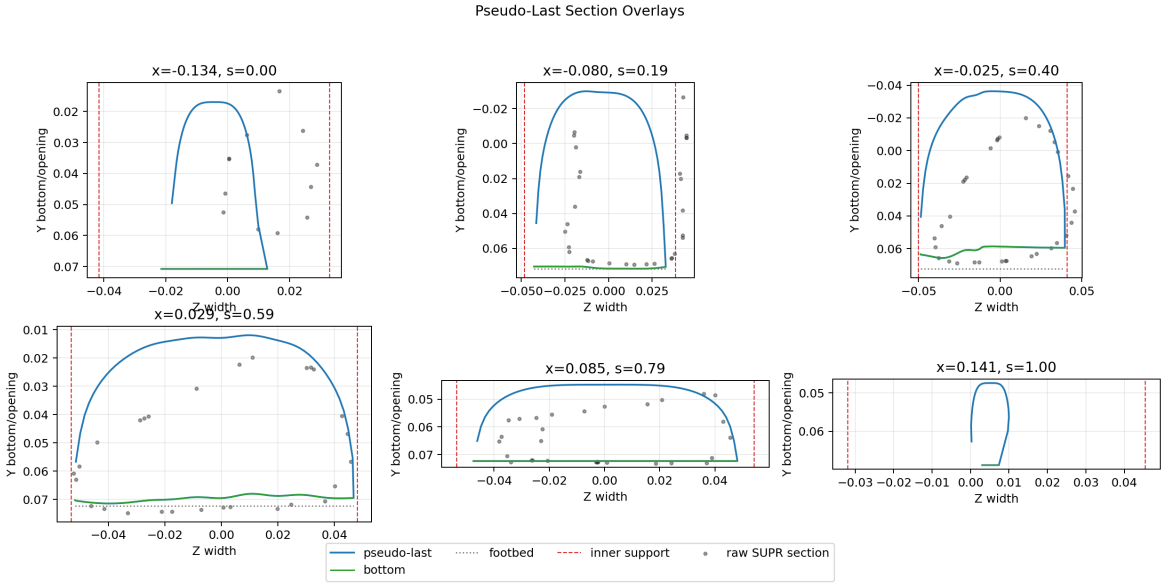

Figure 3: Air Jordan pseudo-last section overlays. Each panel is one Y-Z section along the X length direction.

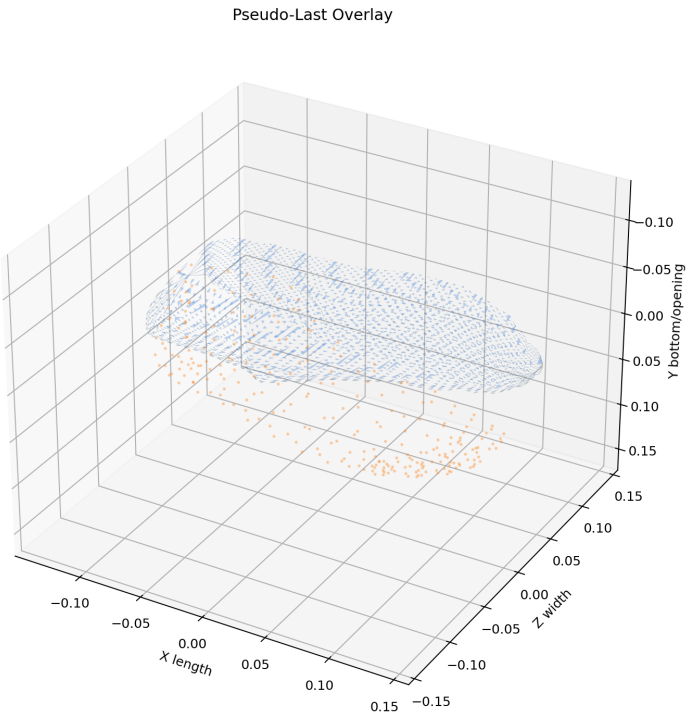

Figure 4: Air Jordan pseudo-last overlay. The output is a watertight section-lofted pseudo-last, not a direct inflated SUPR foot.

7 Part 4: GShell Fields and Where the Prior Enters

7.1 GShell Fields

In `gshell_tets_geometry.py`, GShell maintains several learnable quantities over the tetrahedral grid:

- S_θ: the SDF field. In the current pseudo-last config, `use_sdf_mlp=true`, so this is represented by an MLP.
- M: the mSDF field. In the current pseudo-last config, `use_msdf_mlp=false`, so this is a learnable per-grid tensor.
- D: the deformation field, initialized at zero and clamped to a small fraction of grid spacing.
- material appearance parameters, optimized through image rendering losses.

When `use_sdf_mlp=true`, the SDF MLP is first pretrained to a sphere:

$$S_\theta(\mathbf{p}) \approx \|\mathbf{p}/\texttt{boxscale}\| - \texttt{sphere\_init\_norm}.$$

Here $S_\theta(\mathbf{p})$ is the value predicted by the learned SDF network at point $\mathbf{p}$. The term $\|\mathbf{p}/\texttt{boxscale}\|$ is the normalized distance of the point from the origin. The constant `sphere_init_norm` controls the radius of the initial sphere. This is only an initialization step; it gives the SDF network a simple starting shape before image-based shoe training begins.

The mSDF tensor is randomly initialized approximately around zero:

$$M_i \sim U(0,1) - 0.01,$$

then clamped during training. Here M_i is the mSDF value at tet-grid vertex i, and $U(0,1)$ means a random number sampled between 0 and 1.

7.2 Extraction and Rendering

At each training iteration, `getMesh()` computes:

$$\mathbf{v}_i^{\text{def}} = \mathbf{v}_i + \Delta\mathbf{v}_i,$$

$$S_i = S_\theta(\mathbf{v}_i^{\text{def}}),$$

$$M_i = \texttt{self.msdf}_i.$$

Then GShell extraction is called as:

$$\texttt{gshell\_tets}(\mathbf{v}_i^{\text{def}}, S_i, M_i, \texttt{tets}).$$

Here $\mathbf{v}_i$ is the original tet-grid vertex, $\Delta\mathbf{v}_i$ is the learned deformation of that vertex, and $\mathbf{v}_i^{\text{def}}$ is the deformed grid vertex. S_i is the SDF value at that deformed vertex. M_i is the mSDF value at the same grid vertex. `tets` stores which grid vertices form each tetrahedron. The function `gshell_tets` uses these inputs to extract the current shoe mesh.

The extracted mesh is rendered and compared with the input images. The pseudo-last prior contributes additional regularization losses in `tick()`.

8 Part 5: All Attempts to Use the Pseudo-Last

8.1 Attempt 1: Surface Collision and Containment

The first idea was to use the pseudo-last as a collision object:

$$L_{\text{collision}} = \mathbb{E}_{\mathbf{p}\in\text{shoe surface}}\left[\max(0, m - d_L(\mathbf{p}))^2\right].$$

In this equation, $L_{\text{collision}}$ is the collision penalty, $\mathbb{E}$ means "average over sampled points," and $\mathbf{p}$ is a point sampled on the extracted shoe surface. The value $d_L(\mathbf{p})$ is the stored pseudo-last SDF at that point. With the stored last-SDF convention, $d_L(\mathbf{p}) < 0$ means the shoe surface point is inside the pseudo-last, which is bad. The margin m is a small clearance value. The expression $\max(0, m - d_L(\mathbf{p}))^2$ becomes zero only when the shoe surface point is safely outside the pseudo-last by at least margin m.

The intended effect was that the final visible/open shoe surface should not intersect the pseudo-last.

This was too weak because it acts on the extracted surface after topology has already been decided. If the lower interior has already collapsed or been cut away, a surface collision loss cannot reliably rebuild missing topology.

8.2 Attempt 2: mSDF Keep Loss on Extracted Watertight Vertices

The second idea was to apply a keep loss to watertight shell vertices near the pseudo-last lower support region:

$$L_{\text{keep}} = \mathbb{E}\left[\max(0, m_M - M(\mathbf{p}))^2\right].$$

Here L_{keep} is the mSDF keep penalty, $\mathbf{p}$ is a selected point near the extracted watertight shell, and $M(\mathbf{p})$ is the mSDF value associated with that location. The margin m_M is the small positive target value I want mSDF to exceed. If $M(\mathbf{p}) \geq m_M$, the penalty is zero. If $M(\mathbf{p}) < m_M$, the loss pushes the mSDF upward toward the keep/closed direction.

The intended effect was to make the mSDF positive around support regions.

This was also weak. The selected points were sparse, and many selected mSDF values were already positive. The metric could look satisfied even when the hidden geometry was still bad.

8.3 Attempt 3: Field-Level mSDF Keep Loss

The next idea was stronger: supervise raw tet-grid mSDF values before mesh extraction. This is what "field-level" means here. Instead of selecting vertices from an already extracted mesh, I selected tetrahedral grid vertices:

$$\mathbf{v}_i \in \text{tet grid}.$$

Then I applied:

$$L_{\text{grid keep}} = \frac{\sum_i w_i \max(0, m_M - M_i)^2}{\sum_i w_i}.$$

Here i indexes tet-grid vertices, not extracted mesh vertices. M_i is the raw mSDF value at grid vertex i. w_i is the importance weight for that grid vertex. A sole point may get a larger weight than a sidewall point. The numerator adds the weighted penalties, and the denominator $\sum_i w_i$ normalizes the result so the loss does not become larger just because more grid vertices were selected.

This was algorithmically more reasonable because the field is supervised before GShell extraction happens. However, it still did not visibly fix the interior collapse.

8.4 Attempt 4: Direct mSDF Field Conditioning

I also tried an even stronger idea:

$$M_i^{\text{march}} = M_i^{\text{raw}} + \lambda_{\text{bias}} B_i^{\text{last}},$$

where B_i^{last} is a pseudo-last-derived keep bias. This directly changes the mSDF sent into GShell extraction.

In this equation, M_i^{raw} is the learned mSDF before I modify it, and M_i^{march} is the mSDF actually passed to `gshell_tets`. B_i^{last} is a hand-built bias field from the pseudo-last. It is positive in selected lower pseudo-last support regions and zero elsewhere. λ_{bias} controls how strongly that bias is added. This was stronger than a loss because it changed the extraction input directly instead of merely encouraging the network through gradients.

This also failed visually and caused collapse in the Air Jordan test. The lesson was that forcing mSDF positive near a local last-derived region is not enough by itself. It does not provide a full constructive model of shoe material thickness, and if the selected region is wrong it can make the extraction worse.

9 Part 6: Current Cleaned Cross-Section Pseudo-Ground-Truth Prior

9.1 Current Active Files

The current cleaned implementation lives mainly in:

- `FootShellGaussian/foot_prior/pseudo_last_loss.py`
- `FootShellGaussian/geometry/gshell_tets_geometry.py`
- `FootShellGaussian/train_gshelltet_polycam.py`
- `FootShellGaussian/configs/shoes_mc_pseudolast_xsec_gt_512.json`
- `FootShellGaussian/scripts/train_shoe.sh`

  The current cleaned prior supports:

  `pseudo_last_prior_mode = cross_section`.

Older config flags for collision, fieldkeep, and fieldbias still exist for compatibility, but the cleaned loss module now only supports the cross-section mode.

9.2 Section Coordinates Used by the Prior

For every query point $\mathbf{p} = (x, y, z)$, the prior computes:

$$s_{\text{raw}} = \frac{x - x_0}{x_1 - x_0},$$

$$u_L = \frac{z - c(x)}{a_L(x)},$$

$$u_S = \frac{z - c(x)}{a_S(x)},$$

$$h = B_L(x, u_L) - y,$$

$$\hat{h} = \frac{h}{H(x)}.$$

Here:

- $\mathbf{p} = (x, y, z)$ is the point being tested;
- x_0 and x_1 are the first and last pseudo-last slice positions along the shoe length;
- s_{raw} is normalized length position;
- $c(x)$ is the support-footprint centerline at length x;
- $a_L(x)$ is the pseudo-last half-width at length x;
- $a_S(x)$ is the support-footprint half-width at length x;
- u_L is lateral position normalized by pseudo-last half-width;

- u_S is lateral position normalized by support half-width;
- $B_L(x, u_L)$ is the pseudo-last bottom surface interpolated from `bottom_sections`;
- h is vertical position relative to the pseudo-last bottom;
- $H(x)$ is pseudo-last section height;
- $\hat{h}$ is the height normalized by local last height.

9.3 Dense Cross-Section Supervision Pool

The prior builds a dense point pool by sampling:

$$N_x \times N_y \times N_z$$

points through the pseudo-last cross-section space. Current defaults are:

$$N_x = 64, \qquad N_y = 96, \qquad N_z = 96.$$

This pool is not random in the geometric sense. The code first chooses N_x slice locations along the shoe length. At one fixed x slice, the remaining plane is the Y-Z plane: Y is vertical and Z is left-right width. The code samples a $N_y \times N_z$ grid in that Y-Z plane. Stacking all these 2D grids along the X direction gives the full 3D pool. Each sampled point is then classified into one of several regions.

9.4 Point Classification

Let $C_L(\mathbf{p})$ be the positive-inside pseudo-last cavity field. Here epsilon means the small tolerance band around the pseudo-last surface. Define:

$$\epsilon_L = \texttt{xsec\_surface\_band}.$$

Here $C_L(\mathbf{p})$ is positive inside the pseudo-last cavity and negative outside it. ϵ_L is a small tolerance band around the pseudo-last surface. I use this tolerance because points extremely close to the surface are ambiguous; I do not want to make a hard material/cavity decision exactly on a noisy boundary.

Then:

$$\text{inside cavity} \iff C_L(\mathbf{p}) > \epsilon_L,$$
$$\text{outside cavity} \iff C_L(\mathbf{p}) < -\epsilon_L.$$

The empty cavity mask is:

$$\mathcal{E} = \{\mathbf{p} : 0 \le s_{\text{raw}} \le 1,\ C_L(\mathbf{p}) > \epsilon_L\}.$$

This means $\mathcal{E}$ contains points that lie within the length range of the pseudo-last and are confidently inside the pseudo-last cavity. These are the points I want the learned shoe SDF to classify as empty space.

The lower material/support masks are outside the pseudo-last cavity and near the lower last:

$$\mathcal{S}_{\text{sole}} = \{\mathbf{p} : C_L(\mathbf{p}) < -\epsilon_L,\ h < 0,\ h \ge -d_{\text{sole}},\ |u_S| \le 1 + \delta_S\},$$

This sole set contains points just below the pseudo-last bottom. The condition $h < 0$ says the point is below the last bottom. The condition $h \geq -d_{\text{sole}}$ prevents the selector from going too far downward. The condition $|u_S| \leq 1 + \delta_S$ keeps the point near the support footprint width, with a small slack δ_S.

$$\mathcal{S}_{\text{plantar}} = \{\mathbf{p} : C_L(\mathbf{p}) < -\epsilon_L,\ 0 \leq \hat{h} \leq r_{\text{plantar}},\ |u_L| \leq 1.15,\ |u_S| \leq 1 + \delta_S\},$$

This plantar set contains low points just outside the bottom part of the pseudo-last. The condition $0 \leq \hat{h} \leq r_{\text{plantar}}$ means the point is above the bottom surface but only in a low vertical slice of the last. The condition $|u_L| \leq 1.15$ keeps it near the pseudo-last body rather than far outside the last.

$$\mathcal{S}_{\text{sidewall}} = \{\mathbf{p} : C_L(\mathbf{p}) < -\epsilon_L,\ 0 \leq \hat{h} \leq r_{\text{wall}},\ |u_L| \geq 0.95,\ |u_S| \leq 1 + \delta_S\}.$$

This sidewall set contains low points near the sides of the pseudo-last. The condition $0 \leq \hat{h} \leq r_{\text{wall}}$ keeps the point in the lower portion of the last height. The condition $|u_L| \geq 0.95$ means the point is near the left or right side of the pseudo-last rather than near the center.

The union is:

$$\mathcal{M} = \mathcal{S}_{\text{sole}} \cup \mathcal{S}_{\text{plantar}} \cup \mathcal{S}_{\text{sidewall}}.$$

So $\mathcal{M}$ is the full set of points that the current prior treats as lower shoe material/support. It is not the whole shoe. It is only a local region around the lower pseudo-last, split into sole, plantar, and sidewall parts.

Current defaults are:

$$d_{\text{sole}} = 0.025, \qquad r_{\text{plantar}} = 0.12, \qquad r_{\text{wall}} = 0.45, \qquad \delta_S = 0.05.$$

Here d_{sole} is measured in scene units, while r_{plantar} and r_{wall} are ratios of local last height. The value $\delta_S = 0.05$ means I allow a five-percent support-width slack around the support boundary.

9.5 Loss Terms

The current prior has four active ideas.

Material/support SDF loss. For lower support points $\mathbf{p} \in \mathcal{M}$, the learned shell SDF should be positive:

$$L_{\text{mat}} = \mathbb{E}_{\mathbf{p} \in \mathcal{M}} \left[\max(0, m_{\text{mat}} - S_\theta(\mathbf{p}))^2 \right].$$

Here L_{mat} is the material/support loss. The set $\mathcal{M}$ contains the lower support points selected by the masks above. $S_\theta(\mathbf{p})$ is the learned shell SDF at point $\mathbf{p}$. The margin m_{mat} is the positive SDF value I want these points to reach. If $S_\theta(\mathbf{p}) \geq m_{\text{mat}}$, the point is already confidently inside the learned shoe volume and the penalty is zero. If $S_\theta(\mathbf{p})$ is too small or negative, the loss pushes the SDF upward. In plain words, this says: these lower support points should be treated as shoe material / shell volume.

Empty cavity SDF loss. For pseudo-last cavity points $\mathbf{p} \in \mathcal{E}$, the learned shell SDF should be negative:

$$L_{\text{empty}} = \mathbb{E}_{\mathbf{p} \in \mathcal{E}} \left[\max(0, S_\theta(\mathbf{p}) + m_{\text{empty}})^2 \right].$$

Here L_{empty} is the empty-cavity loss. The set $\mathcal{E}$ contains points inside the pseudo-last cavity. For these points, I want the learned shell SDF $S_\theta(\mathbf{p})$ to be negative by at least margin m_{empty}. If $S_\theta(\mathbf{p}) \leq -m_{\text{empty}}$, the point is already confidently outside shoe material and the penalty is zero. If the SDF becomes positive there, the model is filling the foot cavity with shoe volume, so the loss pushes it downward.

Last surface SDF loss. For sampled pseudo-last surface points $\mathbf{p} \in \partial L$, the shell SDF is weakly encouraged toward zero:

$$L_{\text{surf}} = \text{SmoothL1}(S_\theta(\mathbf{p}), 0).$$

Here L_{surf} is the pseudo-last surface loss. The symbol ∂L means the surface of the pseudo-last. For a point on that surface, the target SDF value is approximately zero because zero is where an implicit surface boundary lives. SmoothL1 is a robust version of an absolute/squared error. This term is weaker than treating the last as exact ground truth. It only asks the learned shell field to be aware of the approximate internal boundary.

Grid mSDF keep loss. For tet-grid vertices that fall in lower material/support regions:

$$L_{\text{mSDF}} = \frac{\sum_i w_i \max(0, m_M - M_i)^2}{\sum_i w_i}.$$

Here L_{mSDF} is the grid-level mSDF keep loss. The index i runs over selected tet-grid vertices. M_i is the raw mSDF value at grid vertex i. m_M is the positive keep margin. w_i is the importance weight for that grid vertex. The expression $\max(0, m_M - M_i)^2$ is zero only when the mSDF is already positive enough. The division by $\sum_i w_i$ turns the weighted sum into a weighted average.

Weights are:

$$w = 1.0 \text{ for sole}, \quad w = 0.9 \text{ for plantar}, \quad w = 0.75 \text{ for sidewall}.$$

This term tries to keep the mSDF positive in local lower support regions. In plain words, it says: if a tet-grid vertex is in a lower support region, GShell should prefer the keep/closed side of the mSDF cut decision there.

Total scheduled loss. The total pseudo-last loss is:

$$L_{\text{xsec}} = \alpha(t) \left(\lambda_{\text{mat}} L_{\text{mat}} + \lambda_{\text{empty}} L_{\text{empty}} + \lambda_{\text{surf}} L_{\text{surf}} + \lambda_{\text{mSDF}} L_{\text{mSDF}} \right).$$

Here L_{xsec} is the total cross-section pseudo-last prior loss. $\alpha(t)$ is the training schedule at iteration t; it controls whether the prior is off, warming up, or fully active. The λ values are loss weights. Larger λ means that term has more influence during optimization. The four inner losses correspond to lower material, empty cavity, approximate pseudo-last surface, and mSDF keep behavior.

The current Air Jordan 512 config uses:

$$\lambda_{\text{mat}} = 20, \quad \lambda_{\text{empty}} = 10, \quad \lambda_{\text{surf}} = 2, \quad \lambda_{\text{mSDF}} = 0.01.$$

These values mean the current experiment gives much stronger pressure to the SDF material/cavity terms than to the mSDF keep term. That choice reflects the lesson that mSDF alone cannot create material volume; the SDF must first learn a meaningful inside/outside structure.

9.6 Gradient Flow

The pseudo-last prior is called inside `GShellTetsGeometry.tick()`. The important code path is:

`pseudo_last_prior_loss(..., shell_sdf_query_fn, grid_msdf)`.

Here `shell_sdf_query_fn` is a function that lets the pseudo-last prior ask the current GShell SDF network for $S_\theta(\mathbf{p})$ at arbitrary points. `grid_msdf` is the raw mSDF tensor on the tet grid.

Therefore, the same prior module can supervise both the learned SDF network and the learned mSDF grid values.

The shell SDF query is:

$$S_\theta(\mathbf{p}) = \texttt{sdf\_net}(\mathbf{p}).$$

Therefore:

- $L_{\text{mat}}, L_{\text{empty}}, L_{\text{surf}}$ send gradients into the SDF MLP;
- L_{mSDF} sends gradients into the learnable grid mSDF tensor;
- image losses still send gradients through rendering and GShell extraction in the normal way.

Figure 5: Example training render strip from the Air Jordan cross-section pseudo-last run. These images check that exterior rendering remains plausible while the interior prior is active.

Figure 6: Example validation render from the Air Jordan cross-section pseudo-last run. Exterior appearance can remain visually reasonable even when hidden interior topology is still wrong, which is why explicit interior diagnostics are required.

10 Important Late Debug Discovery

The largest bug found late was in the cross-section/grid selector. The selector was intended to choose local lower support points near the pseudo-last bottom. Instead, the old logic selected a large vertical column far below the pseudo-last. In other words, the intended region was:

local band just below or around the last bottom,

but the selected region behaved more like:

everything below the last bottom for a long vertical distance.

For the Air Jordan debug case, the old selector produced roughly:

3367 grid mSDF candidates

with a median normalized height:

$$\hat{h}_{50} \approx -3.3264,$$

and some points as low as:

$$\hat{h}_{\min} \approx -16.8485.$$

These values mean that many selected points were multiple section-heights below the pseudo-last bottom. They were not local shoe support points.

After adding the explicit sole-depth cap:

$$h \geq -d_{\text{sole}}, \qquad d_{\text{sole}} = 0.025,$$

the Air Jordan selector became:

229 grid mSDF candidates

with:

$$h_{50} \approx -0.00987, \qquad h_{99} \approx 0.03966, \qquad \text{far-below count} = 0.$$

This bug likely affected earlier experiments. It means that some of the previous mSDF keep and field-level tests were supervising irrelevant points far below the last rather than the actual local support band. Therefore, the negative results from those trials are still useful, but they should be interpreted cautiously. The corrected selector is a more meaningful test of the same idea.

11 What Worked, What Failed, and What I Learned

Experiment	**Intent**	**Outcome / Lesson**
Pseudo-footbed extraction	Estimate a shoe-specific bottom/footbed reference from reconstructed lower geometry.	Useful as plumbing. It provides centerline, support width, and bottom height, but it inherits errors from the reconstructed shoe mesh.
SUPR alignment	Place a neutral anatomical foot inside each shoe.	Works as a reproducible approximation. It does not fit SUPR beta/pose yet, so it is not personalized or shoe-style-specific.

Experiment	Intent	Outcome / Lesson
Section-loft pseudo-last	Build a smoother last-like mesh instead of inflating a literal foot.	Much better than the initial inflated-foot output. Still empirical and not a manufacturing-grade last. Toe spring and style-specific last design remain open.
Surface collision prior	Prevent final surface from intersecting last.	Too weak and too late. If topology is already cut/collapsed, this cannot rebuild it.
Watertight-surface mSDF keep	Push extracted watertight vertices near last to positive mSDF.	Often numerically satisfied without changing hidden geometry. Point set was sparse and post-extraction.
Grid-level mSDF keep	Push raw grid mSDF positive before extraction.	Better placement in the pipeline, but earlier selector bug likely weakened or misdirected it.
Direct mSDF field bias	Add explicit pseudo-last keep bias into mSDF before marching.	Too blunt. It still collapsed visually and did not create true material thickness.
Cross-section pseudo-ground-truth prior	Use dense structured cross-section points for empty cavity, material support, surface boundary, and mSDF keep supervision.	Current cleaned implementation. More interpretable and easier to debug. Still not a true CSG material/cavity model.

12 Current Status and Reality Check

The current implementation is a better organized and more interpretable prior than the earlier losses. It gives GShell explicit supervision for:

- approximate empty interior cavity;
- lower support/sole/sidewall material points;
- approximate pseudo-last boundary;
- mSDF keep direction on local support grid points.

However, it should not be overclaimed. It does not yet guarantee physical shoe wall thickness. GShell still primarily reconstructs surfaces from exterior images, and the pseudo-last prior is a regularization signal, not a true constructive solid model. The shoe may still collapse internally if the exterior image objective and GShell extraction dynamics dominate the interior prior.

The most accurate current claim is:

> I now have a reproducible pseudo-last and a cleaned cross-section prior that gives structured interior supervision to the SDF and mSDF fields.

The important limitation is:

This is still not equivalent to explicitly constructing shoe material as outer volume minus inner cavity.

13 Code Map

File	Role
`foot_prior/support_footprint.py`	Extracts support footprint, centerline, boundaries, and pseudo-footbed heightmap.
`foot_prior/foot_alignment.py`	Creates initial neutral SUPR foot to shoe transform.
`foot_prior/foot_fit_optimizer.py`	Refines foot placement using scale, yaw, pitch, roll, and translation.
`foot_prior/pseudo_last.py`	Builds the section-loft pseudo-last mesh and saves sections, metrics, and diagnostics.
`scripts/run_foot_aware_alignment_pipeline.py`	Runs support footprint extraction, initial alignment, and optimized foot fit.
`scripts/run_pseudo_last_builder.py`	Builds pseudo-last artifacts from aligned foot and support-footbed outputs.
`foot_prior/pseudo_last_loss.py`	Current cleaned cross-section pseudo-ground-truth loss.
`geometry/gshell_tets_geometry.py`	Loads pseudo-last prior, precomputes grid candidates, calls the loss in `tick()`, and passes SDF/mSDF fields to GShell extraction.
`train_gshelltet_polycam.py`	CLI flags, logging, training loop, and pseudo-last statistics.
`configs/shoes_mc_pseudolast_xsec_gt_512.json`	Current 512-resolution cross-section pseudo-last experiment config.
`scripts/train_shoe.sh`	Passes per-shoe pseudo-last SDF and section paths to training when the config enables the pseudo-last prior.

14 Summary

I first built the plumbing needed for an interior prior. From the reconstructed shoe, I estimate a support footprint and pseudo-footbed by using the lower open bottom geometry, projecting it into the X-Z plane, shrinking the usable footbed region inward, and offsetting a smoothed lower support surface toward the interior. I then align a neutral SUPR right-foot template into each shoe using length-based scale and an optimizer over scale, yaw, pitch, roll, and translation.

Using that aligned foot and pseudo-footbed, I build a formula-based pseudo-last with a section-loft method. Each Y-Z section along shoe length estimates last width, height, bottom, arch lift, and toe-box shape from the SUPR foot and support footprint. Those sections are lofted into a watertight pseudo-last.

I then tried several ways to use this pseudo-last in GShell. Simple surface collision and containment losses were too weak because they acted after extraction. mSDF keep losses on extracted vertices were also weak. Field-level mSDF keep and direct mSDF bias were stronger, but still did not solve collapse. The current cleaned version uses cross-section pseudo-ground-truth supervision: points inside the last supervise empty cavity, local lower points outside the last supervise material/support, last surface points weakly supervise an internal boundary, and local tet-grid points supervise mSDF keep direction.

An important bug was found late: the earlier grid selector accidentally chose a large vertical column far below the last instead of a local sole/support band. After adding an explicit sole-depth cap, the Air Jordan grid candidates dropped from thousands of far-below points to a few hundred local candidates. This means some previous failures may have been affected by incorrect point selection. The corrected cross-section prior is the current clean test, but it still should not be claimed as a full CSG or physical thickness model.